

Architectural Design for High-Frequency Football Player Tracking and Augmented Reality Overlay Systems

The development of a computer vision application capable of tracking football players on a live screen and associating persistent graphical overlays—such as jersey numbers or statistical markers—represents a sophisticated engineering challenge at the intersection of high-speed video ingestion, multi-object tracking (MOT) algorithms, and multimodal large language models (LLMs). The specific requirement to allow manual initialization, where a user "sticks" a number to a player which then persists throughout the live broadcast, necessitates a hybrid architecture. This architecture must reconcile the high-frequency requirements of smooth graphical rendering (30 to 60 frames per second) with the deep semantic reasoning of modern AI APIs like Gemini 2.0, which typically operate at lower frame rates for complex spatial analysis.

Foundations of Real-Time Desktop Ingestion for Sports Analytics

To build a tracking system that operates on a live computer screen, the primary technical hurdle is the low-latency capture of the desktop environment. Traditional screenshot utilities are insufficient for the dynamic nature of a football match, where a delay of even 100 milliseconds can cause a graphical overlay to appear disconnected from the player's actual position. High-performance capture must bypass standard windowing abstractions to access the frame buffer directly.

High-Speed Capture Mechanisms and Benchmarking

On the Windows platform, the most efficient method for frame retrieval is the Desktop Duplication API, which provides a direct path to the GPU's desktop texture. Libraries such as DXCam and BetterCam are specifically designed for deep learning pipelines, where maximizing frames per second (FPS) while minimizing CPU utilization is critical. These libraries utilize Windows Waitable Timers to achieve high-resolution frame timing, which is essential for maintaining a constant framerate in the tracking pipeline.

Metric	DXCam	BetterCam	python-mss	Pillow (PIL)
Average Capture Speed	238.79	240.0+	75.87	15.22

(FPS)				
Latency (Standard Deviation)	1.25 ms	< 1.0 ms	0.54 ms	> 16.0 ms
Primary Technology	D3D Desktop Duplication	D3D Duplication + Waitable Timers	Cross-platform screenshot	GDI / BitBlt
Typical Use Case	Deep learning / FPS Games	High-precision video capture	Cross-platform utilities	Basic scripting

The choice of ingestion technology directly influences the "freshness" of the data entering the tracking model. If an application uses a slower library like MSS, it may only achieve 50-70 FPS on a modern Windows machine, which limits the temporal resolution available to the Kalman filter responsible for predicting player movement. In contrast, DXCam allows the system to sample the screen at rates exceeding 200Hz, providing a much denser set of data points for tracking algorithms to handle rapid changes in direction, such as a striker making a sudden cut or a goalkeeper diving for a ball.¹

Streaming Infrastructure and Protocol Conversion

Once the frames are captured from the desktop, they must be formatted into a stream that can be consumed by both local processing units and cloud-based APIs. For applications utilizing the Google Video Intelligence Streaming API, the AIStreamer ingestion library serves as a critical proxy. This library converts standard protocols such as Real-Time Messaging Protocol (RTMP) or Real-Time Streaming Protocol (RTSP) into a bidirectional gRPC connection. The architectural workflow often involves creating a "named pipe" on the local machine where a tool like GStreamer writes the captured desktop frames, which are then read by the AIStreamer proxy and sent to the cloud for analysis.⁴

This pipeline ensures that the video data is delivered with minimal overhead, but it introduces a trade-off. Standard live streaming protocols often prioritize reliability over absolute latency, whereas the user's requirement for a "live" follow-the-player sign demands sub-second responsiveness. To solve this, the application should implement a local "hot path" for tracking

and a cloud-based "cold path" for complex identification and semantic grounding.

Multimodal AI Integration with Gemini 2.0 for Spatial Reasoning

The Gemini 2.0 Flash Multimodal Live API represents a paradigm shift in how AI understands live video. Unlike traditional computer vision models that are trained on specific classes (e.g., "person," "ball"), Gemini offers open-vocabulary spatial reasoning. This means it can understand instructions like "track the player wearing the captain's armband" without specialized re-training.⁶

Spatial Intelligence and Normalized Coordinates

A core feature of the Gemini API for this application is its ability to perform 2D and experimental 3D spatial understanding. When the model processes a video frame, it can return bounding boxes for objects it identifies. These coordinates are normalized to a 1,000 by 1,000 grid, regardless of the actual resolution of the input screen.⁷

Coordinate Component	Range	Origin
x_min / x_max	0 - 1000	Left to Right
y_min / y_max	0 - 1000	Top to Bottom
Depth (z_center)	Experimental Meters	3D Projection
Orientation (Roll/Pitch/Yaw)	Radians	3D Space

The model can detect and localize objects in real-time, providing structured JSON responses that include the labels and box coordinates. For a football application, a prompt might instruct the model: "Identify all players on the field and return their bounding boxes in [y_min, x_min, y_max, x_max] format." This allows the application to map the API's reasoning onto the actual pixels of the user's computer screen.⁷

Limitations of Frame-by-Frame API Calls

Despite its intelligence, the Gemini Live API has specific technical constraints that impact high-speed tracking. The Live API typically processes discrete image frames at a rate of 1 frame per second (FPS). While it is possible to override this sampling rate, doing so significantly increases token consumption and costs, as each second of video at default resolution consumes approximately 300 tokens.¹¹

Furthermore, session lengths for video-and-audio inputs are currently limited (e.g., approximately 2 minutes in certain Firebase AI Logic implementations), though persistent WebSocket connections on Vertex AI can support longer durations.¹² For a live 90-minute football match, the application cannot rely solely on the cloud API for every frame of tracking. Instead, the API should be used for "Identity Anchoring"—periodically verifying which player is which—while a local tracking algorithm handles the frame-to-frame movement on the user's desktop at 60 FPS.¹⁴

Local Computer Vision Pipeline: Detection and Tracking

To satisfy the user's request for an object that "follows" the player, the application must implement a Multi-Object Tracking (MOT) system locally. This system consists of two distinct components: an object detector and a temporal associator.

The YOLO Framework for High-Speed Detection

The YOLO (You Only Look Once) architecture is the gold standard for real-time sports detection. YOLOv8 and its successors are single-stage detectors, meaning they predict bounding boxes and class probabilities in a single pass through the neural network. This allows them to achieve frame rates exceeding 60 FPS on mid-tier hardware, which is the minimum threshold for smooth broadcast overlays.¹⁶

The YOLO architecture is optimized through a three-part structure:

1. **The Backbone:** A series of convolutional layers and CSPLayer (Cross Stage Partial) blocks that extract high-level features from the image while maintaining spatial information.¹⁷
2. **The Neck:** Utilizing a Feature Pyramid Network (FPN), the neck aggregates features from different layers to ensure that small objects (like the football) and large objects (the players) are both detected with high precision.¹⁷
3. **The Head:** The final layers that output the specific bounding box coordinates and confidence scores.

For a football game, the detector is typically trained on datasets like SoccerNet or the DFL-Bundesliga Data Shootout to recognize "player," "referee," "goalkeeper," and "ball".¹⁷ In the user's specific use case, the YOLO model would be responsible for finding all "person" objects

in every frame of the screen capture.

Multi-Object Tracking Algorithms: ByteTrack vs. DeepOCSORT

Once players are detected, the challenge is maintaining their identity across frames. In a crowded football match, players often overlap (occlusion), move unpredictably, or look identical due to team uniforms. This is where tracking algorithms like ByteTrack and DeepOCSORT are employed.

ByteTrack Logic

ByteTrack is a highly efficient MOT algorithm that addresses the problem of "dropped" detections. Most trackers only associate high-confidence detections, but ByteTrack keeps nearly all detection boxes and categorizes them into high-score and low-score buckets. It first associates high-confidence boxes with existing tracklets using Intersection over Union (IoU) and a Kalman filter. Then, it attempts to match the remaining unmatched tracklets with the low-confidence boxes. This allows it to "rescue" players who are partially obscured or blurred during fast motion.¹⁷

DeepOCSORT and Motion Compensation

DeepOCSORT is a robust extension designed for high-speed scenarios with erratic motion. It utilizes a Kalman filter but adds modules for Dynamic Appearance (DA) and Adaptive Weighting (AW). This is particularly useful in football, where a player's trajectory is rarely a straight line. DeepOCSORT corrects the Kalman filter's tendency to drift by prioritizing visual evidence over mathematical prediction when a player makes a sharp turn.¹⁸

Algorithm	MOTA	FPS	Identity Switches	Key Strength
ByteTrack	77.3%	171	558	Extremely fast, rescues low-confidence hits
DeepSORT	61.4%	~40	781	Uses deep appearance embeddings

DeepOCSORT	High	~37	Low	Best for erratic, non-linear motion
StrongSORT	Very High	10-15	Lowest	Most accurate identity maintenance

For the user's application, ByteTrack provides the best balance of speed and accuracy, ensuring the overlay "stays stuck" even when the player is moving quickly.²²

Manual Seeding and Persistent Player Identification

The user's query emphasizes the ability to "manually stick numbers" at the beginning and have the app follow those players. This process is known as "Manual Initialization" or "Seeding" the tracker.

Manual ROI Selection and Feature Extraction

The application should provide a User Interface (UI) where the user can pause the live stream and draw a rectangle around player number 7. This is achieved through the OpenCV selectROI function. Once the region is selected, the system extracts a unique "appearance signature" for that player. This signature often consists of color histograms or deep learning-based embeddings that represent the visual characteristics of that specific player in that specific lighting.²⁶

This manual assignment creates a "Tracklet" with a persistent ID (e.g., ID_7). The tracking algorithm then takes this initial box and, in every subsequent frame, uses the appearance signature and a Kalman filter to find the same player. If the player is briefly hidden behind another player, the tracker uses "Object Rebirth" logic—keeping the ID in a temporary buffer and attempting to re-identify the player when they re-emerge based on their last known velocity and appearance.¹⁷

Automated Re-Identification using Jersey Numbers

While the user requested manual assignment, the app can be made more robust by incorporating automated Jersey Number Recognition (JNR). This serves as a "fail-safe" for when the manual tracker loses its target.

A JNR pipeline involves:

1. **Pose Estimation:** Using a model like ViTPose to find the player's torso and back.¹⁸
2. **Legibility Filtering:** Using a ResNet34 classifier to determine if the jersey number is

currently visible to the camera.²⁹

3. **OCR / STR:** Using a Scene Text Recognition model like PARSeq to read the digits (e.g., "7").²⁹

By integrating JNR, the application can verify that the player currently labeled with the overlay "7" actually has a "7" on their back. If the local tracker gets confused and swaps ID_7 with another player, the JNR system can detect the mismatch and move the overlay back to the correct person.²⁹

Augmented Reality Overlay and User Interface Design

The final layer of the application is the rendering of the "injected objects." These graphics must appear as though they are "pinned" to the players on the screen, updating their position in every frame.

Transparent Window Implementation

To overlay graphics on a live football game playing in another window (e.g., a web browser or a media player), the application must create a "Transparent Overlay Window." This is a specialized GUI window that is frameless, set to be "always on top," and transparent to mouse events so that the user can still interact with the game underneath.³³

In a Python framework like PyQt6, the window is configured with the following parameters:

- `Qt.WindowType.FramelessWindowHint`: Removes the title bar and minimize/maximize buttons.
- `Qt.WindowType.WindowStaysOnTopHint`: Ensures the graphics aren't hidden by the game window.
- `Qt.WidgetAttribute.WA_TranslucentBackground`: Makes the window background invisible.
- `Qt.WidgetAttribute.WA_TransparentForMouseEvents`: Allows clicks to pass through to the application below.³⁴

Overlay Rendering and Temporal Smoothing

The rendering engine (typically OpenCV or a direct GPU canvas) receives the bounding box coordinates $[x, y, w, h]$ from the tracker. To make the object follow the player smoothly, the application applies "Temporal Smoothing." This prevents the overlay from "shaking" due to small variations in the bounding box predictions. A common technique is to use a weighted average of the position over the last 3-5 frames.¹⁸

The visual object itself—such as a circle with the text "7"—is drawn using alpha blending to ensure it looks professional and doesn't completely block the viewer's sight. The OpenCV `cv2.addWeighted` function is used to create semi-transparent overlays, where the alpha value

determines the opacity.³⁸

$$\text{Overlay_Frame} = (1 - \alpha) \cdot \text{Background} + \alpha \cdot \text{Graphic}$$

By adjusting α (e.g., to 0.7), the user can see the player through the label, which is essential during high-action moments where a solid graphic would be distracting.³⁸

Performance and System Architecture

The success of a live tracking app is measured by its latency. The entire pipeline—from screen capture to graphical rendering—must complete in under 33ms to maintain 30 FPS.

Edge vs. Cloud Latency Considerations

For live sports analysis, "Edge Inference" (processing on the local PC) is mandatory for the tracking layer. Sending every frame to the cloud would introduce seconds of latency due to network round-trips and API processing times.¹⁶

Component	Location	Latency	Purpose
Screen Capture	Edge (DXCam)	< 5ms	Data ingestion
Player Detection	Edge (YOLOv8)	10-20ms	Finding all people
Player Tracking	Edge (ByteTrack)	< 2ms	Maintaining IDs
Context Analysis	Cloud (Gemini)	500ms - 2s	High-level reasoning / JNR
Rendering	Edge (PyQt6)	< 5ms	Visualizing overlays

By running the Gemini Live API in a "parallel" or "asynchronous" thread, the application can use the cloud's intelligence to occasionally "fix" the local tracker without slowing down the live 60 FPS feed. This "Frame Correlation" technique ensures that even if the AI takes 1 second to

respond, its insight is applied back to the correct player ID in the local pipeline.¹⁴

Hardware and Scalability

A modern implementation requires a dedicated GPU (e.g., NVIDIA RTX series) to handle the YOLO inference and the screen capture simultaneously. If the user is playing the football game on the same PC, the application must be highly optimized to avoid stealing "draw cycles" from the game itself. This is achieved by using "Video Mode" in capture libraries like DXCam, which only pushes new frames into the buffer when they are rendered by the OS, reducing redundant processing.²

Advanced Capabilities and Future Outlook

Beyond simple player tracking, the infrastructure described above unlocks a suite of "Agentic" capabilities mentioned in the Gemini research. Once the application can track players and understand the field of play, it can begin to generate automated content.

Semantic Commentary and Tactical Analysis

By passing the tracked coordinates and game state to the Gemini 2.0 API, the application can generate live text or audio commentary. For example, the system could identify a "high-press" formation or a "counter-attack" opportunity in real-time, providing the user with professional-level tactical insights that were previously only available to elite coaches.¹⁸

The Gemini API can also be used for "Condition Monitoring" of the players. By analyzing pose estimation data over time, the system can predict fatigue or injury risk, such as identifying a player whose "knee valgus angle" suggests a potential ACL strain.¹⁸

3D Digital Twins and Minimap Generation

With Gemini 2.0's experimental 3D spatial understanding, the application can attempt to project the 2D screen positions onto a 3D model of the pitch. This enables the generation of a "Digital Twin" or a real-time "Minimap," showing the relative positions of all 22 players and the ball from a top-down perspective.⁴⁵

Ethical, Legal, and Compliance Considerations

Operating an AI system that tracks people in live broadcasts carries significant responsibility. Developers must navigate the evolving landscape of digital rights and privacy laws.

Intellectual Property of Broadcasts

Most live football matches are the intellectual property of the leagues or broadcasters. While using an AI to track players for personal analysis might fall under "fair use" in some contexts, distributing an application that overlays third-party graphics on a copyrighted stream can lead to legal action.⁴⁸ Broadcasters often employ watermarking and fingerprinting to track

unauthorized redistribution, and AI-driven "piracy" is a growing concern for rights holders.⁵⁰

Biometric Privacy and Data Rights

Tracking players' movements and identifying them via jersey numbers or facial features involves the collection of biometric data. In jurisdictions like the EU (GDPR) or Illinois (BIPA), this requires strict compliance with data access and consent rules.⁴⁹ Even if the data is "publicly available" on a screen, the systematic tracking and indexing of that data can be subject to privacy regulations, especially if the data is stored in a persistent database.³²

Gemini API Prohibited Use Policy

Users must ensure their application does not violate Google's safety settings or prohibited use policies. This includes avoiding the generation of harmful content and ensuring that the application is not used to "reverse engineer" the underlying models.⁵³ Furthermore, the API terms specify that human reviewers may occasionally review inputs (in the free tier) to improve the models, which should be considered when handling potentially sensitive or private data.⁵³

Conclusion and Strategic Recommendations

The building of an application to track football players and assign following overlays is entirely possible using a combination of the Gemini 2.0 Multimodal Live API and localized Multi-Object Tracking frameworks. The core of the solution lies in a hybrid architecture: using high-performance local libraries like DXCam and YOLOv8 for 60 FPS tracking and rendering, while leveraging the Gemini API for periodic high-level identification, jersey number recognition, and tactical reasoning.

For a successful implementation, it is recommended to:

1. **Prioritize Ingestion:** Use Desktop Duplication-based capture to minimize latency.²
2. **Employ ByteTrack:** Use a modern MOT algorithm that handles occlusion and low-confidence detections.¹⁷
3. **Implement Hybrid Logic:** Use the Gemini Live API for "Identity Anchoring" rather than per-frame tracking to manage costs and latency.¹⁴
4. **Ensure Compliance:** Review the legal implications of tracking public broadcasts and adhering to the API's terms of service.⁴⁹

This system not only fulfills the user's requirement for a manual "sticky" assignment but also provides a foundation for the future of interactive, AI-augmented sports viewing. By turning a flat video feed into a structured data stream, fans and analysts can engage with the game in a truly immersive, 3D-aware environment.

Works cited

1. Python - Fast Screen Capture - Kyle Fu, accessed February 24, 2026, <https://kylefu.me/2023/02/18/python-fast-screen-capture.html>

2. ra1nty/DXcam: A Python high-performance screen capture library for Windows using Desktop Duplication API - GitHub, accessed February 24, 2026, <https://github.com/ra1nty/DXcam>
3. Fastest way to take a screenshot with python on windows - Stack Overflow, accessed February 24, 2026, <https://stackoverflow.com/questions/3586046/fastest-way-to-take-a-screenshot-with-python-on-windows>
4. Stream live videos | Video Intelligence API - Google Cloud Documentation, accessed February 24, 2026, <https://docs.cloud.google.com/video-intelligence/docs/streaming/live-streaming-overview>
5. How-to stream live videos | Video Intelligence API - Google Cloud Documentation, accessed February 24, 2026, <https://docs.cloud.google.com/video-intelligence/docs/streaming/live-streaming>
6. Gemini Live API: Real-time AI for Manufacturing | Google Cloud Blog, accessed February 24, 2026, <https://cloud.google.com/blog/topics/developers-practitioners/gemini-live-api-real-time-ai-for-manufacturing>
7. Bounding box detection | Generative AI on Vertex AI - Google Cloud Documentation, accessed February 24, 2026, <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/bounding-box-detection>
8. 3D Spatial Understanding with Gemini: How AI Learns to See, Point, and Reason Like Humans - Analytics Vidhya, accessed February 24, 2026, <https://www.analyticsvidhya.com/blog/2025/11/3d-spatial-understanding-with-gemini/>
9. cookbook/examples/Spatial_understanding_3d.ipynb at main · google-gemini/cookbook, accessed February 24, 2026, https://github.com/google-gemini/cookbook/blob/main/examples/Spatial_understanding_3d.ipynb
10. 2D spatial understanding with Gemini - Google Colab, accessed February 24, 2026, https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Spatial_understanding.ipynb
11. Video understanding | Gemini API - Google AI for Developers, accessed February 24, 2026, <https://ai.google.dev/gemini-api/docs/video-understanding>
12. Limits and specifications of the Live API | Firebase AI Logic - Google, accessed February 24, 2026, <https://firebase.google.com/docs/ai-logic/live-api/limits-and-specs>
13. Get started with the Gemini Live API using Firebase AI Logic - Google, accessed February 24, 2026, <https://firebase.google.com/docs/ai-logic/live-api>
14. Hi Gemini, Secure the room Building a Real-Time AI-Powered... | by Ben Mizrahi | Google Cloud - Community | Medium, accessed February 24, 2026, <https://medium.com/google-cloud/hi-gemini-secure-the-room-01903d5541f6>
15. Video processing - Best approaches towards analyzing large videos? - Gemini

- API, accessed February 24, 2026,
<https://discuss.ai.google.dev/t/video-processing-best-approaches-towards-analyzing-large-videos/99381>
16. Real-Time Vision AI in Live Sports Analytics? - GetStream.io, accessed February 24, 2026, <https://getstream.io/blog/ai-sports-analytics/>
 17. Darkmyter/Football-Players-Tracking: Football players ... - GitHub, accessed February 24, 2026, <https://github.com/Darkmyter/Football-Players-Tracking>
 18. How Can You Automate Player and Ball Tracking with Vision AI? - GetStream.io, accessed February 24, 2026, <https://getstream.io/blog/ai-ball-player-tracking/>
 19. Deep Learning-Based Football Player Detection in Videos - PMC - NIH, accessed February 24, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC9296282/>
 20. IASC | Free Full-Text | Automatic Team Assignment and Jersey Number Recognition in Football Videos, accessed February 24, 2026, <https://www.techscience.com/iasc/v36n3/51888/html>
 21. How Top AI Multi-Object Trackers Perform in Real-World Scenarios? - Veroke, accessed February 24, 2026, <https://www.veroke.com/insights/how-top-ai-multi-object-trackers-perform-in-real-world-scenarios/>
 22. Comparative Evaluation of SORT, DeepSORT, and ByteTrack for Multiple Object Tracking in Highway Videos, accessed February 24, 2026, <https://vectoral.org/index.php/IJISICS/article/view/97/89>
 23. DeepOCSORT Tracking Method - Emergent Mind, accessed February 24, 2026, <https://www.emergentmind.com/topics/deepocsort>
 24. An Analysis of Kalman Filter based Object Tracking Methods for Fast-Moving Tiny Objects, accessed February 24, 2026, <https://arxiv.org/html/2509.18451v1>
 25. [2509.18451] An Analysis of Kalman Filter based Object Tracking Methods for Fast-Moving Tiny Objects - arXiv, accessed February 24, 2026, <https://arxiv.org/abs/2509.18451>
 26. OpenCV Object Tracking Project | by Amit Yadav - Medium, accessed February 24, 2026, <https://medium.com/@amit25173/opencv-object-tracking-project-29954e0f0418>
 27. Getting Started With Object Tracking Using OpenCV - GeeksforGeeks, accessed February 24, 2026, <https://www.geeksforgeeks.org/computer-vision/getting-started-with-object-tracking-using-opencv/>
 28. A Complete Review of the OpenCV Object Tracking Algorithms - BroutonLab, accessed February 24, 2026, <https://broutonlab.com/blog/opencv-object-tracking/>
 29. mkoshkina/jersey-number-pipeline: A General Framework ... - GitHub, accessed February 24, 2026, <https://github.com/mkoshkina/jersey-number-pipeline>
 30. Jersey Number Recognition - CatalyzeX, accessed February 24, 2026, <https://www.catalyzeX.com/s/Jersey%20Number%20Recognition>
 31. Towards long-term player tracking with graph hierarchies and domain-specific features, accessed February 24, 2026, <https://arxiv.org/html/2502.21242v1>
 32. Automated player identification and indexing using two-stage deep learning

- network - PMC, accessed February 24, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10282031/>
33. How do you create a transparent activity that can overlay the home screen but is not dismissed when the home or back button is pressed? - Stack Overflow, accessed February 24, 2026,
<https://stackoverflow.com/questions/18370785/how-do-you-create-a-transparent-activity-that-can-overlay-the-home-screen-but-is>
 34. Creating a transparent overlay with qt - python - Stack Overflow, accessed February 24, 2026,
<https://stackoverflow.com/questions/25950049/creating-a-transparent-overlay-with-qt>
 35. Inspiration for my Transparent Overlay Program - Qt Forum, accessed February 24, 2026,
<https://forum.qt.io/topic/159057/inspiration-for-my-transparent-overlay-program>
 36. How to make QPainter elements clickable-through using PyQt - Qt Forum, accessed February 24, 2026,
<https://forum.qt.io/topic/127517/how-to-make-qpainter-elements-clickable-through-using-pyqt>
 37. Annotating Sports Footage for Player Tracking AI - DataVLab, accessed February 24, 2026,
<https://datavlab.ai/post/how-to-annotate-sports-footage-for-player-tracking-ai>
 38. Transparent overlays with OpenCV - PyImageSearch, accessed February 24, 2026, <https://pyimagesearch.com/2016/03/07/transparent-overlays-with-opencv/>
 39. How To Make A Streaming Overlay Transparent? - Graphic Design Nerd - YouTube, accessed February 24, 2026,
<https://www.youtube.com/watch?v=VHcHYtG2Es8>
 40. How to use Gemini Live API Native Audio in Vertex AI | Google Cloud Blog, accessed February 24, 2026,
<https://cloud.google.com/blog/topics/developers-practitioners/how-to-use-gemini-live-api-native-audio-in-vertex-ai>
 41. bettercam - PyPI, accessed February 24, 2026,
<https://pypi.org/project/bettercam/>
 42. Integrated AI System for Real-Time Sports Broadcasting: Player Behavior, Game Event Recognition, and Generative AI Commentary in Basketball Games - MDPI, accessed February 24, 2026, <https://www.mdpi.com/2076-3417/15/3/1543>
 43. Revolutionizing Gameplay: How Gemini 2.0 Flash Experimental is Redefining Real Time Commentary | by Sarthak Kesarwani | Google Cloud - Medium, accessed February 24, 2026,
<https://medium.com/google-cloud/revolutionizing-gameplay-how-gemini-2-0-flash-experimental-is-redefining-real-time-commentary-b714c94af5dc>
 44. Build voice-driven applications with Live API | Google Cloud Blog, accessed February 24, 2026,
<https://cloud.google.com/blog/products/ai-machine-learning/build-voice-driven-applications-with-live-api>

45. Sports Data Tracking | Genius Sports, accessed February 24, 2026, <https://www.geniussports.com/data-capture/>
46. Building with Gemini 2.0: Spatial understanding - YouTube, accessed February 24, 2026, <https://www.youtube.com/watch?v=-XmoDzDMqj4>
47. Players Tracking and Identification for Minimap Generation in American Football, accessed February 24, 2026, https://www.researchgate.net/publication/396940923_Players_Tracking_and_Identification_for_Minimap_Generation_in_American_Football
48. Is it legal to broadcast live play-by-play updates of NFL games without permission? - Reddit, accessed February 24, 2026, https://www.reddit.com/r/legal/comments/18ci1h8/is_it_legal_to_broadcast_live_playbyplay_updates/
49. Artificial Intelligence (AI) and Entertainment: How To Protect and Enforce Your Rights in the Digital Age of AI | Illinois State Bar Association, accessed February 24, 2026, <https://www.isba.org/public/guide/ai-entertainment>
50. Saga of American Live Sporting Events, Streaming Platforms, and Cybersecurity Law, accessed February 24, 2026, <https://www.aipla.org/list/innovate-articles/saga-of-american-live-sporting-events-streaming-platforms-and-cybersecurity-law>
51. AI and the Future of Sports Media - Level Law, accessed February 24, 2026, <https://level.law/news/ai-and-the-future-of-sports-media>
52. Privacy vs. Security: The Legal Implications of Using Facial Recognition Technology at Entertainment Venues - New York State Bar Association, accessed February 24, 2026, <https://nysba.org/privacy-vs-security-the-legal-implications-of-using-facial-recognition-technology-at-entertainment-venues/>
53. Gemini API Additional Terms of Service | Google AI for Developers, accessed February 24, 2026, <https://ai.google.dev/gemini-api/terms>